Hardware Synchronization

1. while(LOCK==1);
2. LOCK=1

ENTRY CODE

3. Critical Section

4. LOCK=0

EXIT CODE

Hardware solution for CS problem

- The critical-section problem could be solved simply in a single-processor environment if we could prevent interrupts from occurring while a shared variable was being modified.

- Unfortunately, this solution is not as feasible in a multiprocessor environment. Disabling interrupts on a multiprocessor can be time consuming, since the message is passed to all the processors.

- Many modern computer systems therefore provide special hardware instructions that allow us either to test and modify the content of a word or to swap the contents of two words atomically.

```
boolean test_and_set(boolean *target) {
   boolean rv = *target;
   *target = true;

   return rv;
}
```

```
do {
   while (test_and_set(&lock))
      ; /* do nothing */

      /* critical section */

   lock = false;

      /* remainder section */
} while (true);
```

Mutual Exclusion with Test-and-Set

- Shared data:

  boolean lock = false;

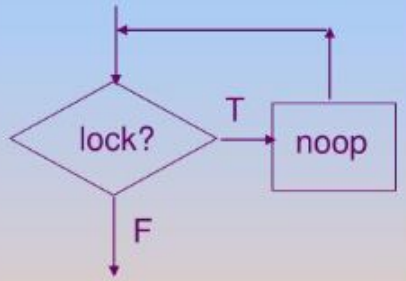
- Process P_i

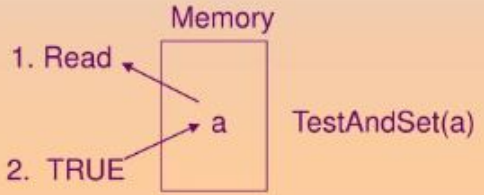
```
do {
   while (TestAndSet(lock)) ;
   critical section
   lock = false;
   remainder section
}
```

1. R.V. 2. TRUE

noop

while(cond) do { };





TestAndSet(a)

** Compare the code readability:
Peterson's algorithm
test & set instruction

```
int compare_and_swap(int *value, int expected, int new_value) {
   int temp = *value;

   if (*value == expected)
      *value = new_value;

   return temp;
}
```

```
do {
   while (compare_and_swap(&lock, 0, 1) != 0)
      ; /* do nothing */

      /* critical section */

   lock = 0;

      /* remainder section */
} while (true);
```